

Contents

- 1. Import der Eingangsdaten
- 2. Parametrisierung des PV-Speichersystems
- 3. Simulation eines PV-Speichersystems mit frühzeitiger und prognosebasierter Batterieladung
- 4. Verarbeitung der Simulationsergebnisse
 - 4.1 Ergebnis-Tabelle erstellen
 - 4.2 Jahresmittlerer Tagesverlauf der Leistungsflüsse
 - 4.3 Tagesverlauf der Leistungsflüsse an einem sonnigen Tag
 - 4.4 Nicht erforderliche Variablen löschen

```
% Skript zur Ausführung des PVprog-Algorithmus

% Simulation mit F5 oder Run starten
```

1. Import der Eingangsdaten

```
load('pvprog_input.mat')

% P_ld(t): elektrische Haushaltslast (load demand) in W
% p_pv(t): spezifische PV-Leistungsabgabe in kW/kWp [0...1]
% time: Zeitstempel im datenum-Format

% Die Grundlage für die Lastdaten stellt ein elektrisches Lastprofil eines
% Haushalts (Nr. 31 der Datenbasis [Tja15]) mit einem Jahresstrombedarf von
% 5010 kWh dar. Das PV-Erzeugungsprofil wurde auf Basis von
% meteorologischen Daten des DWD vom Standort Lindenberg (Brandenburg) des
% Jahres 2013 erstellt. Der spezifische Jahresertrag des simulierten
% PV-Systems beträgt 1004 kWh/kWp. Details des verwendeten
% PV-Simulationsmodells können [Wen13] entnommen werden. Die zeitliche
% Auflösung beider Zeitreihen beträgt 1 min.

% [Tja15] T. Tjaden, J. Bergner, J. Weniger, V. Quaschnig: Repräsentative
% elektrische Lastprofile für Wohngebäude in Deutschland auf 1-sekündiger
% Datenbasis (2015)
%
% [Wen13] J. Weniger, V. Quaschnig: Begrenzung der Einspeiseleistung von
% netzgekoppelten Photovoltaiksystemen mit Batteriespeichern. In: 28.
% Symposium Photovoltaische Solarenergie. Bad Staffelstein, 2013
```

2. Parametrisierung des PV-Speichersystems

```
% Folgende Größen können angepasst werden

P_stc=5; % Nennleistung des PV-Generators in kWp
C_bu=5; % nutzbare Speicherkapazität des Batteriespeichers in kWh
P_bwr=2.5; % Nennleistung des Batteriewechselrichters in kW
p_gfl=0.5; % spezifische Einspeisegrenze in kW/kWp (z.B. 50%-Einspeisebegrenzung des KfW-P
rogramms)
```

3. Simulation eines PV-Speichersystems mit frühzeitiger und prognosebasierter Batterieladung

```

% Wahl der Betriebsstrategie
for bs=1:2; % (1: frühzeitig, 2: prognosebasiert)

    [ a,v,pf,eb,soc ]=pvprog(time,p_pv,P_ld,bs,P_stc,C_bu,P_bwr,p_gfl);

    % Beschreibung der Ergebnis-Variablen: siehe pvprog.m

    % Autarkiegrad (a), Abregelungsverluste (v), Leistungsflüsse (pf),
    % Energiebilanzen (eb) und Ladezustand (soc) je nach Betriebsstrategie
    % speichern
    if bs==1 % frühzeitig
        a_fz=a;
        v_fz=v;
        pf_fz=pf;
        eb_fz=eb;
        soc_fz=soc;

    else % prognosebasiert
        a_pb=a;
        v_pb=v;
        pf_pb=pf;
        eb_pb=eb;
        soc_pb=soc;

    end
end

```

4. Verarbeitung der Simulationsergebnisse

4.1 Ergebnis-Tabelle erstellen

```

Y={'Autarkiegrad in %','Abregelungsverluste in %','PV-Erzeugung in kWh','Stromverbrauch in kWh',
'Direktverbrauch in kWh','Batterieladung in kWh', ...
'Batterieentladung in kWh','Netzeinspeisung in kWh','Netzbezug in kWh','Abregelung in kWh'};

if exist('eb_fz','var') && exist('eb_pb','var')

    X={'fruehzeitig','prognosebasiert'};
    Zfz=[a_fz*100; v_fz*100; eb_fz.E_pv; eb_fz.E_ld; eb_fz.E_du; eb_fz.E_bc; eb_fz.E_bd; eb_fz.E_gf; eb_fz.E_gs; eb_fz.E_ct];
    Zpb=[a_pb*100; v_pb*100; eb_pb.E_pv; eb_pb.E_ld; eb_pb.E_du; eb_pb.E_bc; eb_pb.E_bd; eb_pb.E_gf; eb_pb.E_gs; eb_pb.E_ct];
    Z=[Zfz,Zpb];

elseif bs==1
    X={'fruehzeitig'};
    Z=[ a_fz*100; v_fz*100; eb_fz.E_pv; eb_fz.E_ld; eb_fz.E_du; eb_fz.E_bc; eb_fz.E_bd; eb_fz.E_gf; eb_fz.E_gs; eb_fz.E_ct];

else
    X={'prognosebasiert'};
    Z=[a_pb*100; v_pb*100; eb_pb.E_pv; eb_pb.E_ld; eb_pb.E_du; eb_pb.E_bc; eb_pb.E_bd; eb_pb.E_gf; eb_pb.E_gs; eb_pb.E_ct];

end

Results=array2table(round(Z*10)/10,'RowNames',Y,'VariableNames',X);
disp(Results)

```

4.2 Jahresmittlerer Tagesverlauf der Leistungsflüsse

```
if exist('eb_fz','var') && exist('eb_pb','var')

for bs=1:2

    if bs==1
        pf=pf_fz;
    else
        pf=pf_pb;
    end

ini = zeros(1440,1);
pfm = struct('P_pvm',ini,'P_ldm',ini,'P_dum',ini,'P_bcm',ini,'P_bdm',ini,'P_gfm',ini,'P_gsm',ini,'P_ctm',ini);
varm = fieldnames(pfm);
var = {'P_pv','P_ld','P_du','P_bc','P_bd','P_gf','P_gs','P_ct'};
for i=1:8
    pfm.(varm{i}) = mean(reshape(pf.(var{i}),1440,[],2),2);
end

fig=figure;

barplt=[...
    pfm.P_gfm,...
    pfm.P_bcm,...
    pfm.P_dum,...
    pfm.P_ctm,...
    pfm.P_pvm*-1,...
    pfm.P_dum*-1,...
    pfm.P_bdm*-1,...
    pfm.P_gsm*-1,...
    ]/1000;

dt=time(2)-time(1);
h=bar(time(1:1440),barplt,'stacked','LineStyle','none','barwidth',1.0);
hold on;

h(1).FaceColor = [0.8000    0.8000    0.8000];
h(2).FaceColor = [0.4667    0.6745    0.1882];
h(3).FaceColor = [1.0000    0.8000         0];
h(4).FaceColor = [0         0         0];
h(5).FaceColor = 'none';
h(6).FaceColor = h(3).FaceColor;
h(7).FaceColor = [0.2000    0.4000         0];
h(8).FaceColor = [0.3137    0.3137    0.3137];

ax=gca;
set(gcf,'color','w');
datetick('x',15)

ax.XLim=[time(1) time(1441)];
ax.YLabel.String = 'Leistung in kW';
ax.XLabel.String = ' ';

l=legend([h(7) h(2) h(3) h(1) h(8) h(4)],{'Batterieentladung','Batterieladung','Direktverb  
rauch','Netzeinspeisung',...
    'Netzbezug','Abregelung'},'Location','northwest');
legend('boxoff')
```

```

if bs == 1
title({'Jahresmittlerer Tagesverlauf der Leistungsflüsse';'frühzeitige Batterieladung'},'FontWeight','normal');
else
title({'Jahresmittlerer Tagesverlauf der Leistungsflüsse';'prognosebasierte Batterieladung'},'FontWeight','normal');
end

fonsize=9; set(findall(fig,'-property','FontSize'),'FontSize',fonsize); set(findall(fig,'-property','FontName'),'FontName','Verdana');ax.YLabel.FontSize=fonsize;
l.FontSize=8.5;

ax = gca;
ax.YTickLabel = strrep(ax.YTickLabel, '.', ',');
set(gca,'Layer','top')
hold off;

fig.Units = 'normalized';
if bs == 1
    pos = fig.Position;
    fig.Position(1) = 0.01; fig.Position(2) = 0.92-pos(4);
elseif bs == 2
    pos2 = fig.Position;
    fig.Position(1) = 0.02+pos(3); fig.Position(2) = 0.92-pos(4);
end
end
end

```

4.3 Tagesverlauf der Leistungsflüsse an einem sonnigen Tag

```

% Tag des Jahres für die Grafik
dsel=203;

if exist('eb_fz','var') && exist('eb_pb','var')

for bs = 1:2

    if bs==1
        pf=pf_fz;
    else
        pf=pf_pb;
    end
for i=1:8
    tmp = pf.(var{i});
    pf.(var{i}) = tmp((dsel-1)*1440+1:dsel*1440);
end

fig = figure;

barplt=[...
    pf.P_gf,...
    pf.P_bc,...
    pf.P_du,...
    pf.P_ct,...
    pf.P_pv*-1,...
    pf.P_du*-1,...
    pf.P_bd*-1,...
    pf.P_gs*-1,...
]/1000;

```

```

dt=time(2)-time(1);
h=bar(time(1:1440),barplt,'stacked','LineStyle','none','barwidth',1.0);
hold on;

h(1).FaceColor = [0.8000    0.8000    0.8000];
h(2).FaceColor = [0.4667    0.6745    0.1882];
h(3).FaceColor = [1.0000    0.8000         0];
h(4).FaceColor = [0         0         0];
h(5).FaceColor = 'none';
h(6).FaceColor = h(3).FaceColor;
h(7).FaceColor = [0.2000    0.4000         0];
h(8).FaceColor = [0.3137    0.3137    0.3137];

ax=gca;
set(gcf,'color','w');
datetick('x',15)

ax.XLim=[time(1) time(1441)];
ax.YLabel.String = 'Leistung in kW';
ax.XLabel.String = ' ';
ax.YLim=[-2.5 P_stc];

l=legend([h(7) h(2) h(3) h(1) h(8) h(4)],{'Batterieentladung','Batterieladung','Direktverb
rauch','Netzeinspeisung',...
      'Netzbezug','Abregelung'},'Location','northwest');
legend('boxoff')

if bs == 1
title({'Verlauf der Leistungsflüsse an einem sonnigen Tag';'frühzeitige Batterieladung'},'
FontWeight','normal');
else
title({'Verlauf der Leistungsflüsse an einem sonnigen Tag';'prognosebasierte Batterieladun
g'},'FontWeight','normal');
end

set(findall(fig,'-property','FontSize'),'FontSize',fonsize); set(findall(fig,'-property','
FontName'),'FontName','Verdana');ax.YLabel.FontSize=fonsize;
l.FontSize=8.5;

frame=1:1440;
P_pvframe=p_pv((dsel-1)*1440+1:dsel*1440)*1000*P_stc;
if max(P_pvframe)>p_gfl*P_stc*1000
frame=frame(P_pvframe>p_gfl*P_stc*1000);
p=plot(time(frame),repmat(p_gfl*P_stc,size(frame)),'LineWidth',1.5,'Color',[0.8000    0.20
00         0]);
text(time(ceil(frame(end)+20)),p_gfl*P_stc,[num2str(p_gfl*100),'%-Grenze'],'FontName','Ver
dana','FontSize',8.5,'Color',[0.8000    0.2000         0])
end

ax.YTickLabel = strrep(ax.YTickLabel, '.', ',');
set(ax,'Layer','top')
hold off;

fig.Units = 'normalized';
if bs == 1
    pos3 = fig.Position;
    fig.Position(1) = 0.01; fig.Position(2) = pos(2)-0.08-pos3(4);
elseif bs == 2
    pos4 = fig.Position;
    fig.Position(1) = 0.02+pos(3); fig.Position(2) = pos2(2)-0.08-pos4(4);

```

```
end  
end  
end
```

4.4 Nicht erforderliche Variablen löschen

```
clearvars -except time p_pv P_ld P_stc C_bu P_bwr p_gfl pf_pb eb_pb a_pb v_pb pf_fz eb_fz  
a_fz v_fz Results
```